# Implementação N.03 — Caminho mínimo com o menor número de arestas

Teoria dos Grafos · Luca Lodi · outubro de 2026

## 1. Problema

Dado um grafo direcionado com pesos positivos, uma origem *s* e um destino *t*, encontrar o caminho de menor custo de *s* até *t*. Se existir mais de um caminho com esse custo, deve ser escolhido o que tem o menor número de arestas. O programa mostra o comprimento (custo) do caminho, o número de arestas e a lista de vértices.

## 2. Solução

- **Representação:** *forward star*. As arestas ficam em dois vetores (`to` e `weight`) agrupadas por origem; as arestas de *u* ocupam as posições `pointer[u]` até `pointer[u+1]-1`. A montagem conta as arestas de cada vértice e faz uma soma acumulada, então o arquivo não precisa estar ordenado. Memória O(n + m).
- **Algoritmo:** Dijkstra comparando o par **(custo, arestas)** em vez de só o custo. Um vértice *v* é atualizado quando `dist[u] + w < dist[v]` ou quando `dist[u] + w == dist[v]` e `edges[u] + 1 < edges[v]`. A fila de prioridade (`PriorityQueue`, heap binário) usa a mesma ordem: menor custo e, no empate, menos arestas.
- **Por que funciona:** como todo peso é positivo, andar por uma aresta sempre aumenta o par (custo, arestas). O par se comporta como um peso composto e o Dijkstra continua correto: quando *t* sai da fila, o custo é mínimo e, entre os caminhos com esse custo, o número de arestas também é mínimo. A busca termina nesse momento.
- **Caminho:** o vetor `parent` guarda o vértice anterior; o caminho é montado de *t* para trás.
- **Complexidade:** O((n + m) log n) por consulta e O(n + m) para ler o arquivo.

#### Entrada, uso e exemplo

Arquivo texto: na primeira linha `n m`; depois `m` linhas `origem destino peso`, com vértices de 1 a n (linha sem peso vale 1). Execução: `java ShortestPath <arquivo> <origem> <destino>`, ou sem argumentos para digitar no terminal.

exemplo.txt — dois caminhos de custo 4 de 1 até 5

```
5 5
1 2 3
1 3 1
2 5 1
3 4 1
4 5 2
```

java ShortestPath exemplo.txt 1 5

```
Grafo: exemplo.txt (n = 5, m = 5)
Origem: 1 | Destino: 5
Comprimento do caminho minimo: 4
Numero de arestas: 2
Caminho: 1 -> 2 -> 5
```

Um Dijkstra comum devolve `1 → 3 → 4 → 5` (3 arestas): o vértice 4 (custo 2) sai da fila antes do 2 (custo 3) e chega primeiro em 5. Com o desempate, quando o 2 é processado o caminho de mesmo custo e 2 arestas substitui o anterior.

## 3. Metodologia dos testes

- **Grafo aleatório:** n vértices e 5n arestas com origem e destino sorteados (sem laços). Grau de saída médio 5; os caminhos têm poucas arestas.
- **Grafo em grade:** lado × lado vértices, cada um ligado aos vizinhos de cima, baixo, esquerda e direita (m ≈ 4n). Os caminhos têm centenas de arestas, como em uma malha de ruas.
- **Tamanhos:** n = 1 mil, 10 mil, 100 mil e 1 milhão (grade: $32^2$, $100^2$, $316^2$ e $1000^2$). Pesos inteiros sorteados de 1 a 10. Sementes fixas: os grafos são gerados sempre iguais por `java Benchmark`.
- **Consultas:** 20 pares (origem, destino) sorteados por grafo, depois de 3 consultas de aquecimento da JVM. Médias de custo e arestas consideram só as consultas em que existe caminho.

- **Eficácia:** o resultado é comparado com o Dijkstra comum (o mesmo código sem o desempate) e conferido com um Bellman-Ford por rodadas: na rodada *k* ele calcula o menor custo usando no máximo *k* arestas, então a primeira rodada que atinge o custo ótimo dá o menor número de arestas possível. Como o Bellman-Ford é O(n·m), a conferência roda até n = 100 mil.
- **Eficiência:** tempo de leitura (arquivo → *forward star*) e tempo médio por consulta, medidos com `System.nanoTime`.
- **Ambiente:** AMD Ryzen 7 5825U, 15,4 GB de RAM, Windows 11, Java 21.0.11 (Temurin).

## 4. Resultados

**Tabela 1 — Grafos aleatórios (m = 5n)**

| Grafo | | | | Eficácia | | | | | Eficiência (ms) | |
|---|---|---|---|---|---|---|---|---|---|---|
| n | m | alcançáveis | custo médio | arestas (desempate) | arestas (Dijkstra comum) | consultas com menos arestas | conferidas c/ Bellman-Ford | leitura | consulta (desempate) | consulta (comum) |
| 1.000 | 5.000 | 20/20 | 18,2 | **5,70** | 5,70 | 0 | 20/20 | 74 | **0,71** | 0,63 |
| 10.000 | 50.000 | 19/20 | 25,1 | **7,47** | 7,68 | 3 | 20/20 | 129 | **2,85** | 2,41 |
| 100.000 | 500.000 | 19/20 | 31,8 | **10,26** | 10,63 | 3 | 20/20 | 380 | **46,14** | 41,11 |
| 1.000.000 | 5.000.000 | 20/20 | 34,6 | **12,25** | 12,40 | 1 | — | 3.554 | **786,13** | 655,97 |

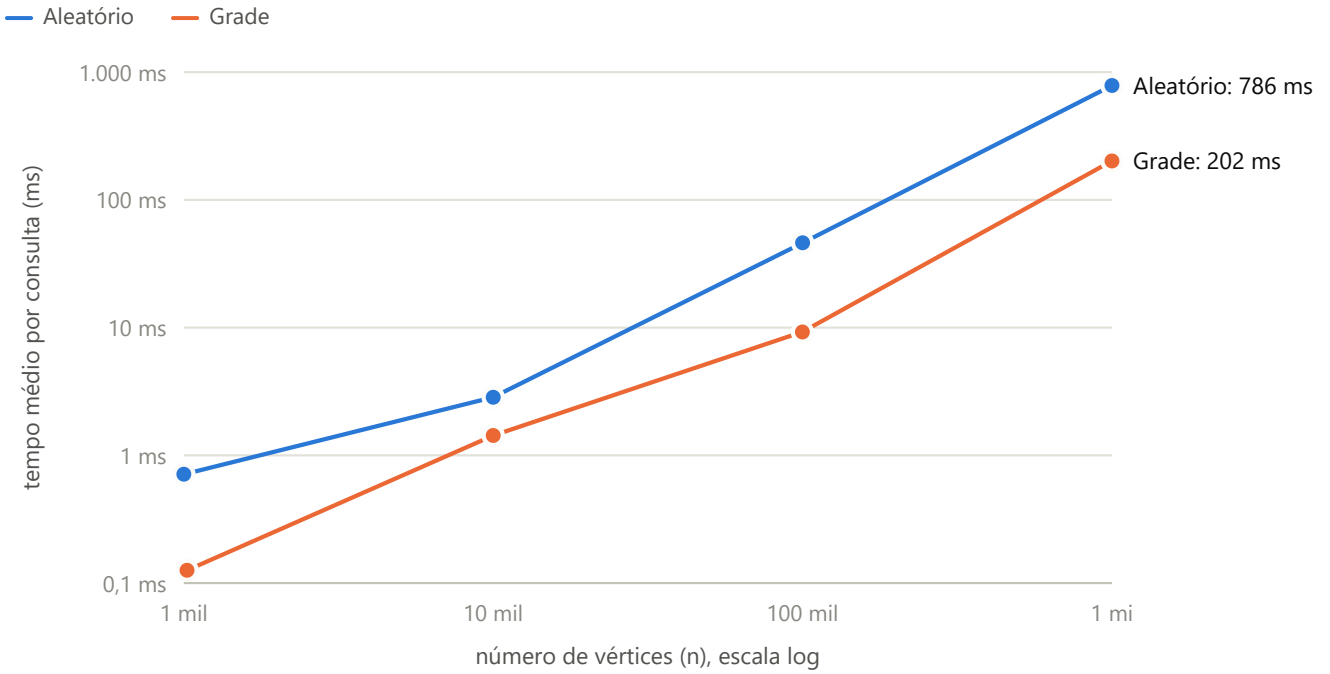
Arestas: média do número de arestas do caminho encontrado. "Consultas com menos arestas": vezes em que o desempate achou um caminho de mesmo custo e menos arestas que o Dijkstra comum. "—": grafo grande demais para o Bellman-Ford.

**Tabela 2 — Grafos em grade (m ≈ 4n)**

| Grafo | | | | Eficácia | | | | | Eficiência (ms) | |
|---|---|---|---|---|---|---|---|---|---|---|
| n | m | alcançáveis | custo médio | arestas (desempate) | arestas (Dijkstra comum) | consultas com menos arestas | conferidas c/ Bellman-Ford | leitura | consulta (desempate) | consulta (comum) |
| 1.024 | 3.968 | 20/20 | 90,5 | **26,90** | 27,20 | 2 | 20/20 | 54 | **0,13** | 0,11 |
| 10.000 | 39.600 | 20/20 | 230,6 | **75,30** | 76,10 | 3 | 20/20 | 45 | **1,43** | 1,24 |
| 99.856 | 398.160 | 20/20 | 508,9 | **172,30** | 172,80 | 3 | 20/20 | 262 | **9,24** | 8,18 |
| 1.000.000 | 3.996.000 | 20/20 | 1.921,7 | **662,90** | 665,30 | 13 | — | 1.806 | **201,73** | 169,86 |

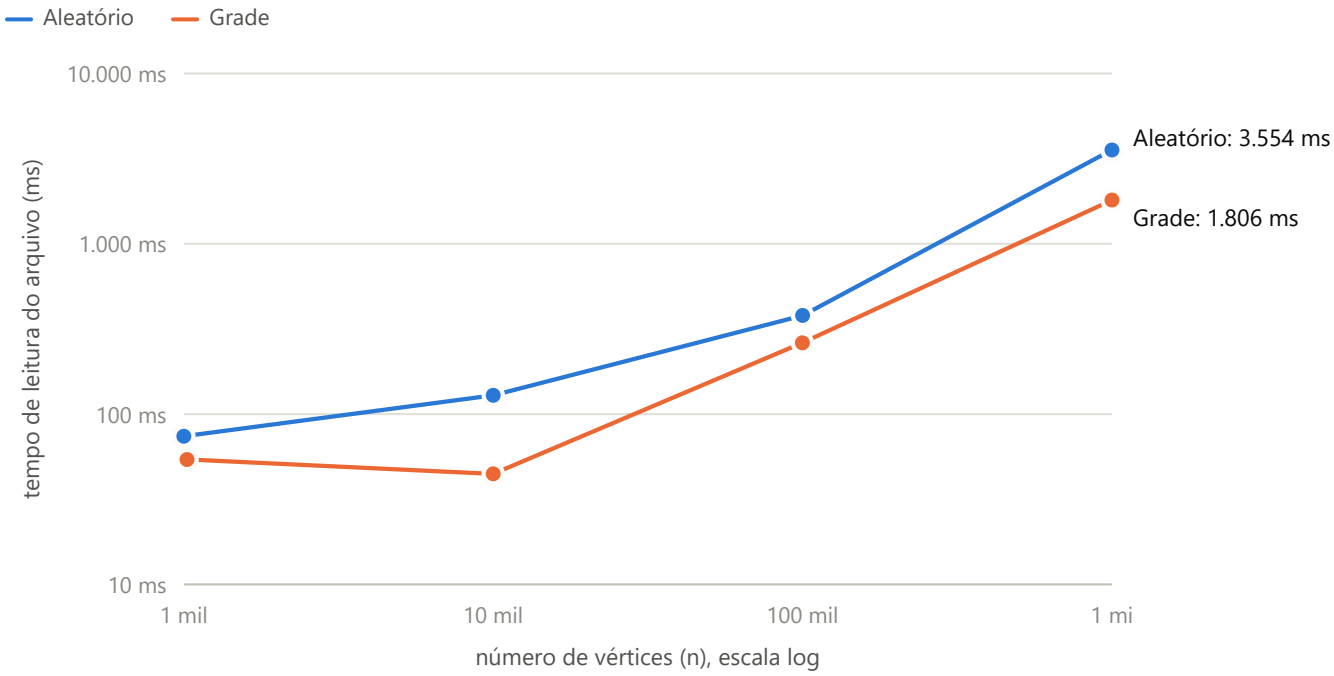
Mesmas colunas da Tabela 1. Na grade, n é o quadrado do lado ($32^2$, $100^2$, $316^2$, $1000^2$).

**Figura 1 — Tempo médio por consulta (com desempate)**



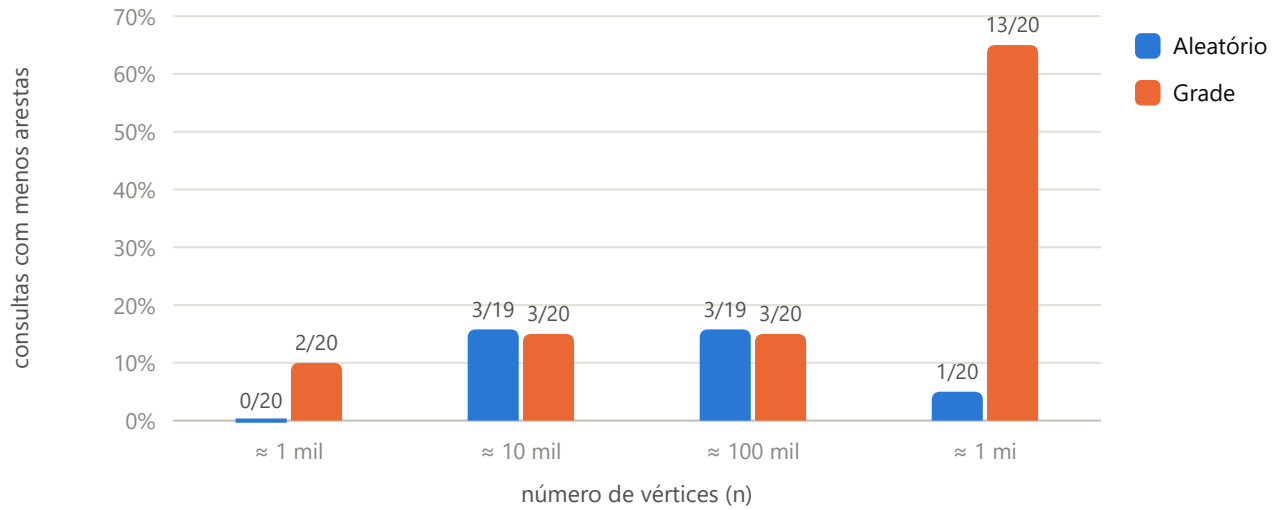

Eixos em escala logarítmica. Uma reta com inclinação 1 indica crescimento proporcional a n.

**Figura 2 — Tempo de leitura do arquivo**




Leitura do texto e montagem da *forward star*. Eixos em escala logarítmica.

**Figura 3 — Consultas em que o desempate encontrou um caminho com menos arestas**




Percentual das consultas com caminho; o rótulo mostra a contagem. Em todas as demais, os dois métodos acharam caminhos com o mesmo número de arestas.

## 5. Análise

### Eficácia

- Nas 120 consultas conferidas (6 grafos × 20), o custo e o número de arestas foram iguais aos do Bellman-Ford, incluindo as consultas sem caminho.
- Nas consultas conferidas, o Dijkstra comum também chegou ao custo ótimo; a diferença está só em qual caminho ele escolhe. Em 28 das 158 consultas com caminho, ele devolveu um caminho com mais arestas que o necessário.
- O desempate faz mais diferença na grade, principalmente na maior: caminhos longos (centenas de arestas) têm mais rotas alternativas que empatam no custo. Na grade de 1 milhão de vértices, 13 das 20 consultas tiveram caminho mais curto em arestas. No grafo aleatório os caminhos têm em média de 6 a 12 arestas e os empates são raros.

### Eficiência

- O tempo por consulta cresce um pouco mais que linearmente com n (Figura 1), como esperado para O((n + m) log n), e fica abaixo de 1 segundo mesmo com 1 milhão de vértices e 5 milhões de arestas.
- O grafo aleatório é mais lento que a grade com o mesmo n: tem mais arestas (5n contra 4n) e os vizinhos ficam espalhados na memória, enquanto na grade os vizinhos têm números próximos e aproveitam melhor o cache.
- O desempate deixou as consultas de 12% a 20% mais lentas que o Dijkstra comum, por causa das comparações extras e das reinserções na fila quando aparece um caminho de mesmo custo com menos arestas. A complexidade não muda.
- A leitura do arquivo cresce aproximadamente de forma linear com m e, para uma única consulta, custa mais que o próprio Dijkstra (3,6 s para 5 milhões de arestas). Depois de carregado, o grafo pode responder várias consultas.